

Trip Fare Estimation

Ben-Gurion University of the Negev
Department of Industrial Engineering and Management
Data Science Methods and Applications
Course No.: 364.1.1121

Submitted by:

Omer Gabay
Netanel Rachum

Instructor:

Trivikram Muralidharan

Submission Date:

17.6.26



This project addresses the business problem of Trip Fare Estimation - predicting the approximate cost of a taxi ride before the booking is made. In this business scenario, Gett operates in the New York City taxi market and is required to provide app users with a reliable fare estimate before they press the "Book Ride" button.

An accurate fare estimate is important for both passengers and drivers: an overestimate may cause passengers to abandon the booking, while an underestimate may hurt driver satisfaction and service profitability.

The data used in this project comes from the NYC TLC Yellow Taxi Trip Records dataset, which contains historical records of yellow taxi rides in New York City. As specified in the assignment guidelines, data from 2024 and 2025 is used for model building and training, while data from January-February 2026 is reserved exclusively for model evaluation and performance demonstration. This split simulates a realistic scenario in which the model is built on historical data and tested on future data that was not used during training.

The target variable is defined as:

$\text{total_fare_amount} = \text{total_amount} - \text{tip_amount}$

That is, the total trip cost excluding the tip. The tip is excluded because it is not a guaranteed component of the pre-trip fare - it depends on the passenger's personal decision after the ride is completed. The model therefore aims to estimate the predictable cost of the trip itself, rather than a payment component that varies by individual behavior.

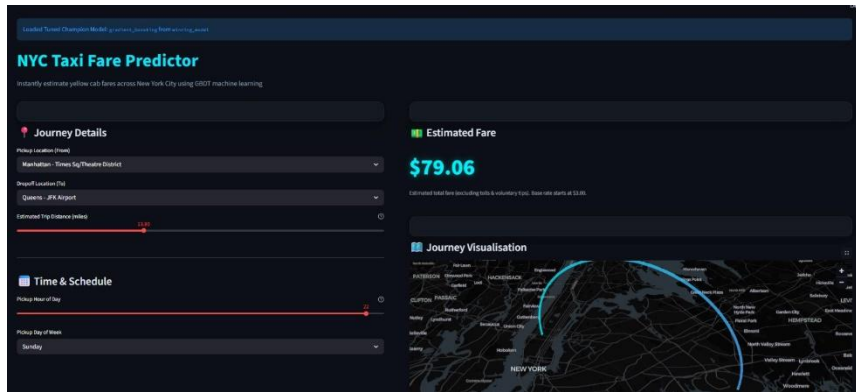
The features permitted as model inputs are those available at booking time, before the ride begins: pickup zone (PULocationID), dropoff zone (DOLocationID), trip distance (trip_distance), time of day, and day of week. By contrast, monetary columns such as fare_amount, extra, mta_tax, tolls_amount, congestion_surcharge, airport_fee, tip_amount, and total_amount cannot be used as features, since they are generated during or after the ride. Using them as inputs would constitute Data Leakage - a situation in which the model receives information that would not be available at inference time.

The project encompasses a full Machine Learning pipeline covering the following stages: data loading and validation checks; data cleaning including removal of duplicates and replacement of impossible values; exploratory data analysis; feature engineering and train-test splitting; training and comparing multiple models; hyperparameter tuning and champion model selection; error analysis; drift detection across time periods; and experiment tracking using Weights & Biases. The purpose of this report is to summarize the key steps taken, explain the decisions made throughout the process, and demonstrate how the pipeline built here addresses the business problem of estimating taxi fares prior to booking.

[GitHub link](#)

[WANDB link](#)

1.1: Stream lit UI - NYC Taxi Fare Predictor



2. Exploratory Data Analysis and Key Insights

The Exploratory Data Analysis (EDA) stage was performed in order to understand the structure of the NYC TLC Yellow Taxi dataset, identify data quality issues, and extract initial insights relevant to the business problem of pre-booking fare estimation. Since the goal of the project is to estimate the fare before the ride takes place, the analysis focused on variables that can explain fare behavior using information available at booking time, such as trip distance, pickup and dropoff zones, time of day, day of week, and special trip types such as airport rides.

2.1 Data Quality and Fare Distribution

A first important finding from the EDA was that the raw dataset contains several data quality issues, including negative fare values, unrealistic trip distances, invalid timestamps, and cases where the dropoff time appears earlier than the pickup time. Such records are not meaningful for fare prediction and may distort both the analysis and the trained model. Therefore, validation and cleaning were necessary before feature engineering and model training.

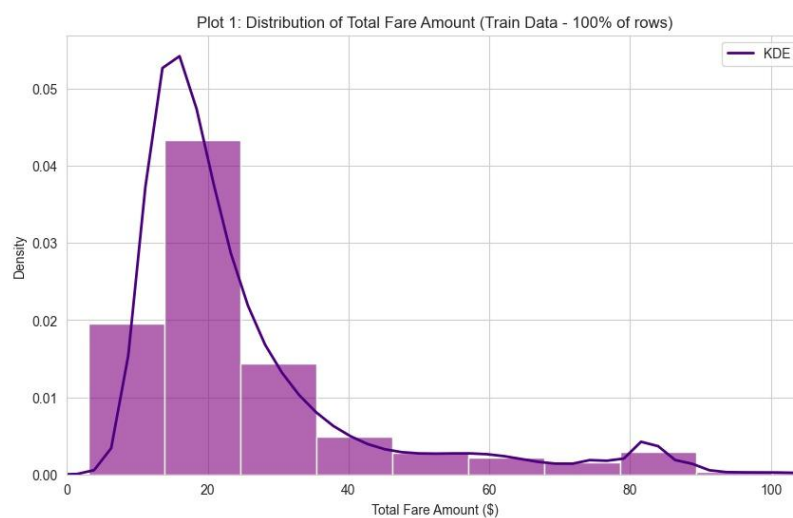


Figure 2.1: Distribution of total_fare_amount.

The distribution of the target variable shows that most taxi rides are concentrated within a reasonable fare range, while extreme values and invalid observations require careful preprocessing. This supports the need for data cleaning before training the model.

This analysis also supported the definition of the target variable. The dataset includes both total_amount and tip_amount, but tips are not part of the predictable fare before the ride. In practice, the passenger decides whether to tip only after the ride is completed, and tips may also be recorded differently depending on the payment method. Therefore, the target variable was defined as:

$$\text{total_fare_amount} = \text{total_amount} - \text{tip_amount}$$

This definition better matches the business goal of estimating the expected trip fare before booking, rather than predicting the passenger's final payment behavior.

2.2 Relationship Between Trip Distance and Fare

Trip distance was one of the most important variables identified during the EDA. As expected, longer trips generally lead to higher fares, since taxi pricing is strongly related to the distance traveled. However, the relationship is not perfectly linear: trips with similar distances can still have different prices because of airport fees, congestion surcharges, time-based extras, tolls, and route-specific differences.

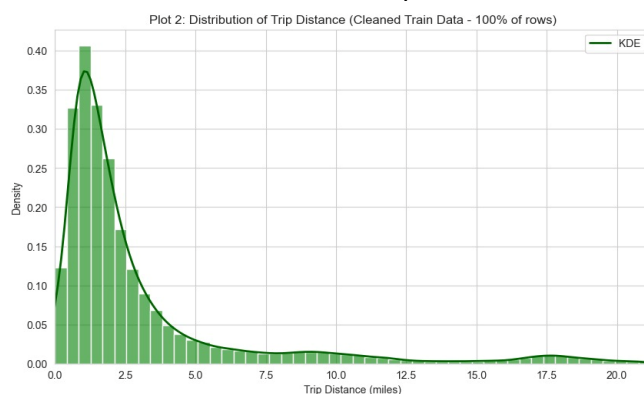
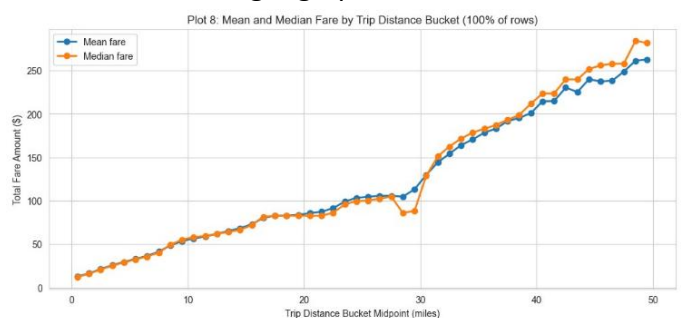


Figure 2.2: Mean and Median Fare by Trip Distance Bucket.

The plot shows a clear and consistent positive relationship between trip distance and fare amount, confirming that distance is the primary driver of taxi pricing. At the same time, the divergence between mean and median fare at longer distances indicates that distance alone longer trips show greater fare variability due to airport fees, tolls, and -is not sufficient route-specific differences, supporting the need for additional location-based and fare-rule features. This finding directly supported the use of trip_distance as a core feature, as well as the creation of haversine_distance, which provides an additional geographic distance measure between pickup and dropoff zones



2.3 Time-Based Fare Patterns

Time-related patterns were also examined during the EDA. Pickup hour, day of week, weekend indicator, holiday indicator, and time-of-day bucket are all available at booking time and may help explain fare variation. For example, nighttime rides and peak-hour rides may include additional charges or reflect different traffic and demand conditions.

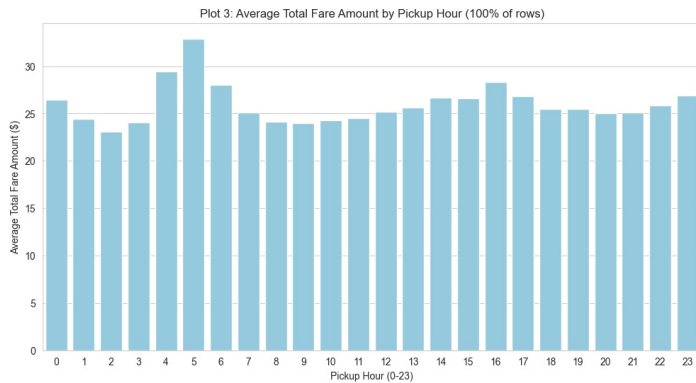


Figure 2.3: Average fare by pickup hour or day of week

The time-based analysis shows that fare behavior changes across different hours or days. These patterns justify the creation of temporal features such as `pickup_hour`, `day_of_week`, `is_weekend`, `is_holiday`, and `time_of_day_bucket`.

Although the later feature importance analysis showed that time-based features were less dominant than distance and location features, they were still included because they are available at booking time and reflect real business conditions such as peak-hour and nighttime pricing.

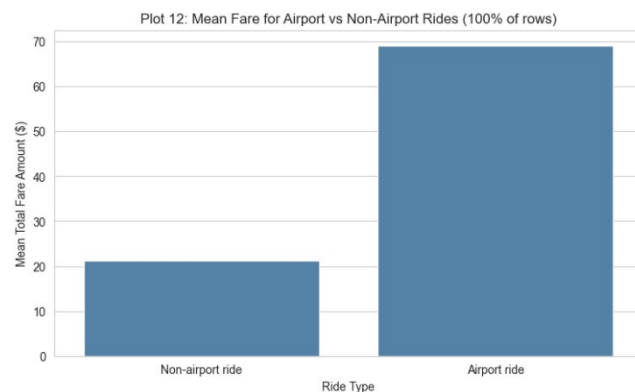
2.4 Airport and Location-Based Fare Behavior

Location was another central factor in fare estimation. Pickup and dropoff zones help capture price variation that cannot be explained by distance alone. In particular, airport-related rides often follow different pricing patterns due to special fare rules, airport fees, longer distances, and more diverse route structures.

Figure 2.4: Fare behavior for airport versus non-airport trips.

Airport-related trips show different fare behavior compared with regular city trips. This supports the use of location-based fare logic, including airport indicators, pickup and dropoff zones, and estimated airport fee components.

This insight was especially important for the feature engineering stage. It motivated the inclusion of `PULocationID`, `DOLocationID`, and location-based fare rules such as airport-related fees and congestion surcharge logic. It also explains why airport trips later appeared as one of the higher-error segments in the model improvement analysis.



3. Feature Engineering and Final Features

The feature engineering phase was one of the most central stages of this project. The model's objective is not to predict a fare after the ride has ended, but to estimate the trip cost before the user presses the "Book Ride" button. Therefore, all features had to be based solely on information that is known or estimable at booking time - such as pickup zone, dropoff zone, trip distance, time of day, and day of week. The goal of this phase was to transform raw data into a set of variables that faithfully represents the business logic of taxi fare pricing, without relying on any information generated during or after the ride.

3.1 Preventing Data Leakage

Particular care was taken to prevent Data Leakage. Monetary columns present in the NYC TLC dataset - such as `total_amount`, `tip_amount`, `fare_amount`, `extra`, `mta_tax`, `tolls_amount`, `congestion_surcharge`, `airport_fee`, and `payment_type` - were not used as model features, since they represent information unavailable to the user prior to booking. Using these columns would have given the model direct access to actual fare components, resulting in an unrealistically optimistic performance estimate. That said, `total_amount` and `tip_amount` were used solely to compute the target variable:

$$\text{total_fare_amount} = \text{total_amount} - \text{tip_amount}$$

After computing the target, the original monetary columns were excluded from the model input. The tip was excluded because it depends on the passenger's personal decision after the ride, and therefore cannot be part of a pre-booking fare estimate.

3.2 Key Engineered Feature: `est_total_fare_without_tolls`

The most significant feature engineered in this project is `est_total_fare_without_tolls`. This is a domain-based feature designed to approximate the expected trip fare excluding tolls, constructed from fare components that can be estimated before the ride takes place. As shown in Figure 1 below, this feature received the highest importance score by a very large margin (0.7124), indicating that the model relies heavily on structured fare logic rather than on blind pattern recognition from zone IDs or time variables alone.

The feature was computed as the sum of several estimated fare components:

$$\begin{aligned} \text{est_total_fare_without_tolls} = & \\ & \text{est_base_fare} \\ & + \text{est_congestion_surcharge} \\ & + \text{est_extra} \\ & + \text{est_airport_fee} \\ & + \text{est_improvement_surcharge} \\ & + \text{est_mta_tax} \end{aligned}$$

The `est_base_fare` component was calculated using rate-based logic. For trips between JFK Airport and Manhattan, a fixed flat rate of \$70 was applied. For all other trips, the base fare was estimated using:

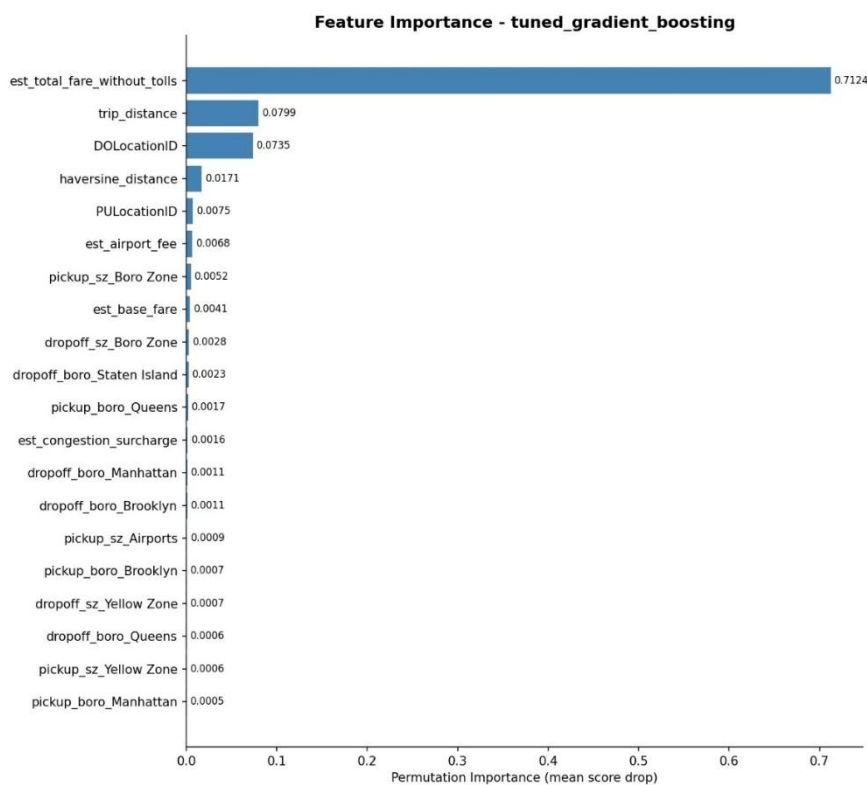
$$\text{est_base_fare} = 3.0 + 3.50 \times \text{trip_distance}$$

Additional fare components were estimated as follows. The `est_congestion_surcharge` was set to \$2.50 when either the pickup or dropoff zone falls within lower Manhattan, and \$0 otherwise. The `est_extra` component was determined by time of day: \$1.00 for nighttime trips, \$2.50 for weekday peak-hour trips, and \$5.00 for JFK trips during peak hours. The `est_airport_fee` was estimated based on whether the trip involved a major airport — JFK, LaGuardia, or Newark. Finally, two fixed components were added: `est_improvement_surcharge` at \$1.00 and `est_mta_tax` at \$0.50.

It is important to emphasize that this is an approximation built from domain logic, not a direct copy of actual fare columns from the dataset. Despite being a highly predictive feature, it does not constitute Data Leakage, as it is computed exclusively from permitted variables such as distance, time, and pickup/dropoff zones.

3.3 Feature Importance Analysis

Figure 1: Permutation Feature Importance - tuned_gradient_boosting model



The feature importance plot reveals a clear hierarchy among the engineered features. The top five features account for the vast majority of the model's predictive power, while the remaining features - including borough-level encodings such as `pickup_boro_Queens`, `dropoff_boro_Manhattan`, and zone-type indicators such as `pickup_sz_Airports` - contribute smaller but meaningful signals. This pattern confirms that the model's core prediction mechanism is driven by fare estimation logic and geographic distance, with location-specific features providing additional refinement.

3.4 Final Feature Set

The table below summarizes the five most important features in the final model:

Feature	Type	Importance	Explanation
est_total_fare_without_tolls	Domain-based estimate	0.7124	Approximated fare using TLC rate logic (base fare, surcharges, airport fees) — computed from permitted inputs only
trip_distance	Distance	0.0799	Estimated trip distance, used as a proxy for routing engine output at booking time
DOLocationID	Geographic	0.0735	Drop-off zone — captures destination-specific fare patterns and special rate zones
haversine_distance	Geographic	0.0171	Straight-line distance between pickup and dropoff zones — adds spatial context beyond route distance
PULocationID	Geographic	0.0075	Pick-up zone — part of the location-based fare rules family (airport fees, congestion zones)
pickup_hour, day_of_week, is_weekend, is_holiday, time_of_day_bucket	Temporal	Low	Time-based features available at booking time — capture peak-hour surcharges, nighttime pricing, and demand patterns

4. Code Modularity & Model Selection and Hyperparameter Tuning

4.1: Project Codebase Structure

The codebase is organized around a single entry point (pipeline.py) that orchestrates six modular source files under src/, each with a single responsibility: data cleaning, feature engineering, model training, evaluation, drift detection, and experiment tracking

```
/nyc-tlc-project-dsma-course
|
/data —┴─
/models —┴─
/notebooks —┴─
/outputs —┴─
|
/src —┴─
download_data.py —┴─ |
cleaning.py —┴─ |
features.py —┴─ |
models.py & tuning.py —┴─ |
evaluation.py & validation.py —┴─ |
drift_detection.py —┴─ |
drift_mitigation.py —┴─ |
experiment_tracking.py —┴─ |
|
pipeline.py —┴─
streamlit_app.py —┴─
requirements.txt —┴─
environment.yml —┴─
```

After completing the data cleaning and feature engineering stages, several regression models were trained and evaluated in order to identify the most suitable model for the taxi fare estimation task. The goal was not only to obtain the lowest prediction error, but also to select a model capable of capturing non-linear relationships between distance, location, time, and the engineered fare-related features. Throughout the model selection and tuning process, Weights & Biases (W&B) was used for experiment tracking. Each run logged the model type, hyperparameter configuration, and evaluation metrics, enabling a structured comparison between candidate models and supporting the selection of the final champion model.

The evaluated models included both simple baseline models and more advanced machine learning models. Baseline models - Linear Regression and Ridge-type linear models - were used to establish an initial reference point. In addition, several non-linear and tree-based models were tested, including Decision Tree, Random Forest, and Gradient Boosting.

Alternative model families such as Neural Network and Linear SVM were also evaluated in order to compare different modeling approaches.

The main evaluation metric used for model comparison was **Mean Absolute Error (MAE)**. MAE was selected as the primary metric because it is directly interpretable in the business context of fare estimation: it represents the average absolute difference, in dollars, between the predicted fare and the actual fare. A lower MAE therefore means that the model provides estimates closer to the real trip cost. Additional metrics - RMSE and MAPE - were also monitored to provide a more complete picture of model performance.

The initial model comparison showed that tree-based ensemble models outperformed simpler approaches. The Gradient Boosting model achieved the best initial performance with an MAE of 5.47, compared with 5.65 for Random Forest, 7.19 for Decision Tree, 8.68 for Linear Regression, 8.76 for SVM, and 12.19 for the Neural Network. These results confirm that the fare estimation problem involves non-linear patterns and feature interactions that simpler linear models cannot adequately capture.

The table below summarizes the results of all evaluated models

Model	MAE	MAPE	RMSE
Tuned Gradient Boosting	5.31	16.01%	12.81
Engineered Gradient Boosting	5.47	16.58%	12.96
Engineered Random Forest	5.65	17.34%	13.24
Engineered Decision Tree	7.19	24.89%	14.07
Engineered Linear Regression	8.68	30.78%	15.76
Engineered SVM	8.76	30.84%	15.77
Engineered Neural Network	12.19	43.92%	19.49

Table 3.1: Model comparison based on W&B evaluation results. The tuned Gradient Boosting model achieved the lowest MAE, MAPE, and RMSE across all evaluated models, and was therefore selected as the final champion model.

Following the initial comparison, hyperparameter tuning was performed in two stages. First, a **Random Search** was used to explore a broad range of model families and hyperparameter combinations, including the number of estimators, maximum tree depth, minimum samples per leaf, learning rate, and L2 regularization. The objective of the search was to minimize MAE across all candidate configurations.

In the second stage, a focused **Grid Search** was applied around the most promising model family. Since Gradient Boosting achieved the strongest initial performance, the tuning process concentrated on refining this model specifically, searching over parameters such as `n_estimators`, `max_depth`, `learning_rate`, `min_samples_leaf`, and `l2_regularization`. This two-stage strategy allowed the project to first explore a broad modeling space and then refine the best-performing model in a more controlled and efficient manner.

The final selected model - the **tuned Gradient Boosting** - achieved an MAE of **5.31**, MAPE of **16.01%**, and RMSE of **12.81**, representing a measurable improvement over the initial Gradient Boosting baseline. The model is well-suited to this business problem because taxi fare is influenced by multiple interacting factors - trip distance, pickup and dropoff location, airport-related fees, congestion surcharges, and time-based effects - and Gradient Boosting is capable of combining these signals effectively through its sequential tree-building mechanism.

Overall, the model selection process confirmed that the tuned Gradient Boosting model was the strongest candidate across all evaluated configurations. It was therefore selected as the final champion model for the Trip Fare Estimation task.

5. Model Improvement Loops: Error Analysis, Drift Detection and Mitigation

5.1. Model Improvement Loops

5.1.1 Error Analysis

After selecting the tuned Gradient Boosting model as the champion model, an error analysis was performed in order to understand where the model performs well and where it tends to make larger prediction errors. Instead of relying only on the overall MAE, the errors were analyzed across meaningful business segments: trip distance, airport versus non-airport rides, and rush-hour versus off-peak rides.

Error by Trip Distance

The first analysis examined model error by trip distance quartile. As shown in Figures 5.1.1 and 5.1.2, the model performs best on medium-length trips and worst on long-distance trips. The highest error was observed in Q4 (the longest-distance quartile), with an MAE of \$7.89, while Q2 achieved the lowest MAE of \$3.37. Q1 and Q3 had intermediate MAE values of \$5.02 and \$4.93 respectively. This pattern indicates that long trips are inherently more difficult to predict accurately, likely due to greater route variability, potential tolls, airport-related detours, and the larger absolute fare values involved.

Figure 5.1.1: MAE by Trip Distance Quartile

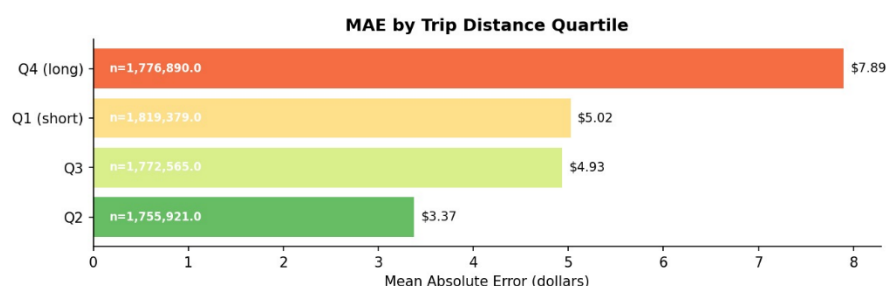
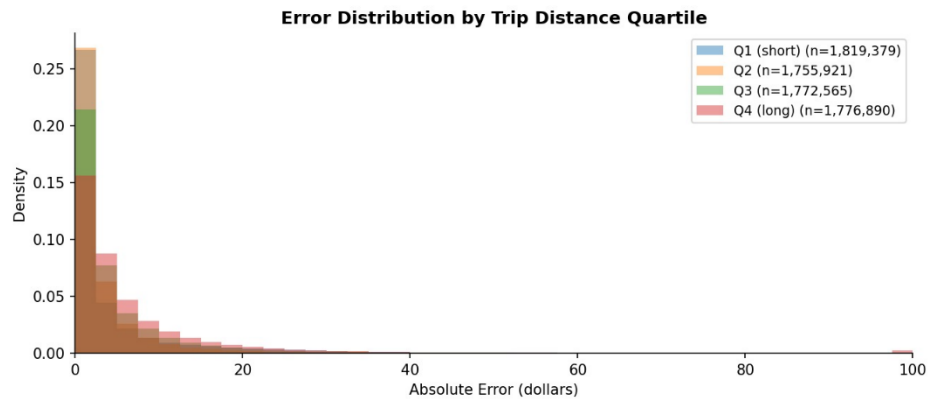


Figure 5.1.2: Error Distribution by Trip Distance Quartile



Error by Airport vs. Non-Airport Trips

A second important finding emerges from the comparison between airport and non-airport trips. As shown in Figures 5.1.3 and 5.1.4, airport trips had a significantly higher MAE of \$9.18, compared with \$5.06 for non-airport trips. This result is consistent with the business logic of the problem: airport rides involve special fare rules, fixed-rate segments, airport fees, longer distances, and more diverse routing patterns — all of which increase prediction difficulty.

Figure 5.1.3: MAE by Airport vs. Non-Airport

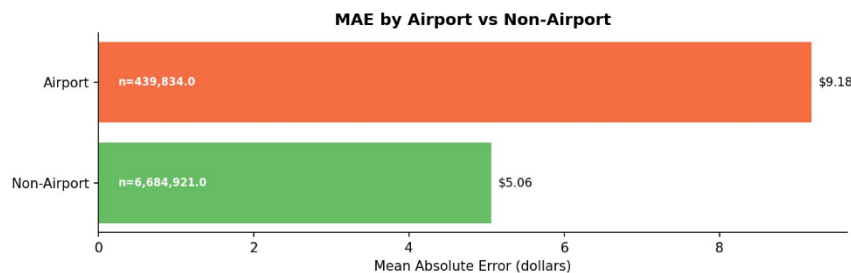
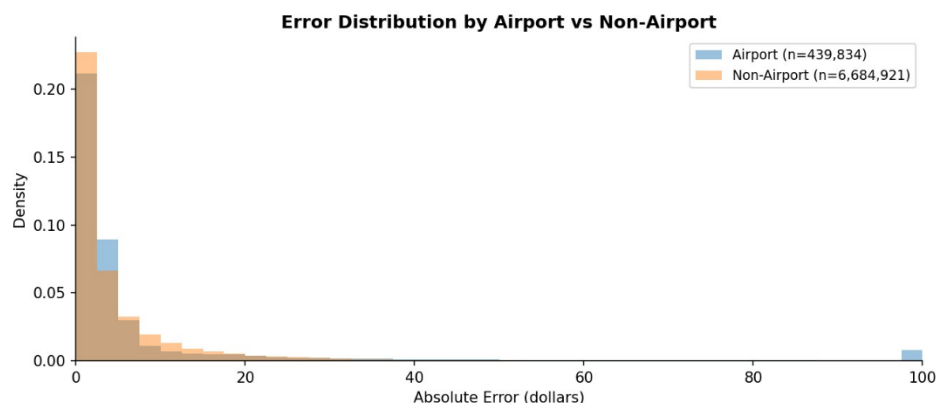


Figure 5.1.4: Error Distribution by Airport vs. Non-Airport



Error by Rush Hour vs. Off-Peak

The third analysis compared rush-hour and off-peak trips. As shown in Figures 5.1.5 and 5.1.6, the difference here was smaller than in the previous two analyses: rush-hour trips had an MAE of \$5.59, compared with \$5.14 for off-peak trips. This suggests that time-related

effects exist but are less dominant than distance and airport-related effects in driving prediction error.

Figure 5.1.5: MAE by Rush Hour vs. Off-Peak

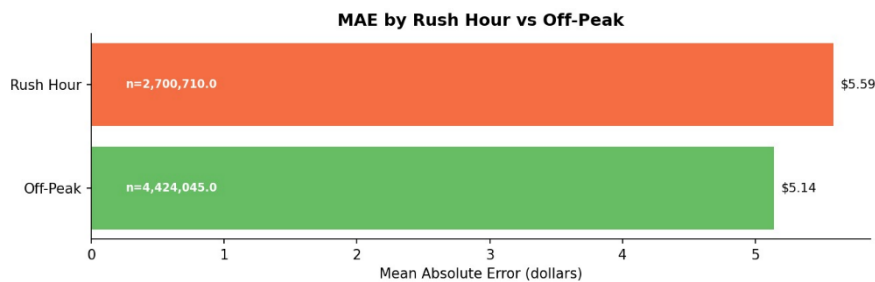
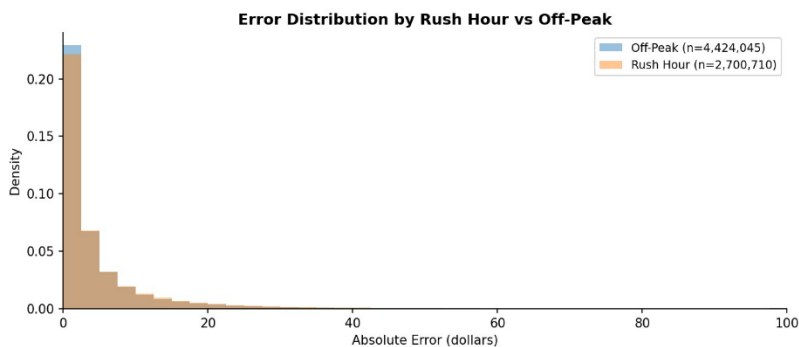


Figure 5.1.6: Error Distribution by Rush Hour vs. Off-Peak



Summary of Error Analysis

Overall, the error analysis reveals that the model performs well across most common trips, but prediction errors increase in more complex scenarios — particularly long-distance rides and airport-related trips. These findings provide clear direction for future model improvements: refining the airport fare estimation logic, handling long-distance trips more carefully, and monitoring high-error segments beyond the global MAE metric.

5.2 Drift Detection

2025 with -Drift detection was performed in order to compare the training data from 2024 the hold-out evaluation data from January-February 2026. This step was important because a model that performs well on historical data may become less reliable if the distribution of future data changes over time. The analysis focused on both feature drift and target drift.

The main variables examined were trip_distance, PULocationID, DOLocationID, and the target variable total_fare_amount. These variables were selected because they are directly related to the fare estimation task and represent the main types of information used by the model: distance, geography, and the final fare target.

Two drift measures were considered: the Kolmogorov-Smirnov (KS) test and the Population Stability Index (PSI). The KS test produced statistically significant p-values for all examined variables. However, this result should be interpreted carefully, because the dataset contains millions of rows, and with such a large sample size even very small distributional differences can become statistically significant. Therefore, PSI was used as the main practical drift indicator.

The results are summarized in the table below:

Feature / Metric	PSI	KS Statistic	KS p-value	Practical Drift
trip_distance	0.0028	0.0159	0.0	No
PULocationID	0.0006	0.0051	0.0	No
DOLocationID	0.0004	0.0029	0.0	No
total_fare_amount	0.0077	0.0435	0.0	No

Table 5.1: Drift detection summary comparing the 2024–2025 training data with the January–February 2026 hold-out test set. Although the KS test indicates statistically significant differences, all PSI values are far below the 0.10 threshold, suggesting no practically significant drift.

The PSI results showed no practically significant drift across all examined variables. All PSI values are far below the common 0.10 threshold, indicating that the distributions remained stable between the training period and the 2026 evaluation period.

The target variable also remained stable. The mean fare in the training data was \$24.43, compared with \$22.85 in the 2026 test set. Although this indicates a small shift in the average fare, the PSI value of 0.0077 confirms that the overall target distribution did not drift in a practically meaningful way.

Overall, the drift detection results indicate that the 2026 evaluation set is sufficiently similar to the 2024–2025 training data. This supports the reliability of the model evaluation and suggests that the model did not face substantial distribution shift during the hold-out test period.

5.3 Drift Mitigation

Based on the drift detection results, no immediate drift mitigation action was required. Although the KS test indicated statistically significant differences between the 2024-2025 training data and the January-February 2026 test data, the PSI values for the examined variables were all far below the common 0.10 threshold. Therefore, the detected differences were not considered practically significant, and the model could still be evaluated reliably on the 2026 hold-out data.

However, drift mitigation remains an important part of maintaining the model in a real production environment. Since taxi demand, trip patterns, airport activity, congestion behavior, and fare rules may change over time, the model should not be treated as a one-time static solution. Instead, it should be monitored periodically using new incoming trip data.

The first recommended mitigation strategy is **periodic retraining**. Future versions of the model should incorporate newly collected taxi trip records, especially if monitoring shows changes in key variables such as trip_distance, PULocationID, DOLocationID, or total_fare_amount. Retraining would allow the model to learn updated travel patterns and maintain accurate fare predictions over time.

The second mitigation strategy is **targeted monitoring of high-error segments** identified in the error analysis. Long-distance trips and airport-related trips showed higher prediction errors, and therefore these segments should be monitored separately from the global MAE. If the error in these groups increases over time, it may indicate that the fare estimation logic or the location-based features require refinement.

A third mitigation action is **maintaining the domain-based fare logic** used in the engineered features. Since `est_total_fare_without_tolls` was the most important feature in the final model, changes in TLC fare rules, airport fees, congestion surcharges, or fixed-rate policies should be reflected in the feature engineering code. Updating this logic is especially important because the model relies heavily on it for accurate fare estimation.

If future drift is detected, the full pipeline should be rerun: data validation, cleaning, feature engineering, model training, hyperparameter tuning, W&B comparison, and champion model selection, making drift mitigation an integral part of a continuous machine learning lifecycle rather than a one-time correction.

Overall, while no immediate mitigation was needed based on the current PSI results, the project defines a clear maintenance strategy: monitor key features and high-error segments, update fare-rule logic when needed, and retrain the model periodically using newer data.

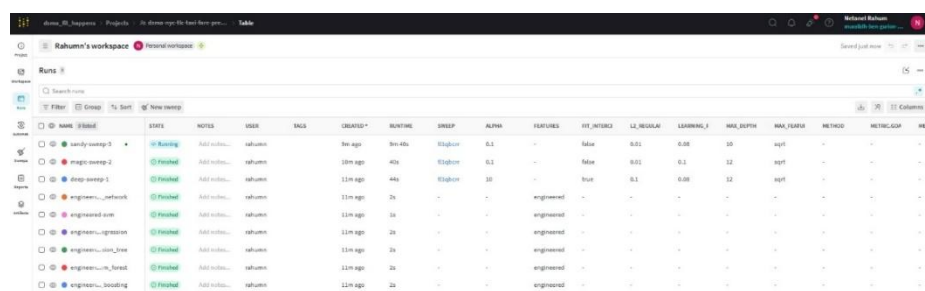
6. Experiment Tracking with Weights & Biases

Weights & Biases (W&B) was used as the experiment tracking platform throughout this project. Each run logged the model type, hyperparameter configuration, and evaluation metrics — including MAE, RMSE, and MAPE — enabling structured comparison across all candidate models.

The tracked runs covered the full experimentation process: baseline models, engineered models, tuned models, and the final champion evaluation. The tuned Gradient Boosting model was selected based on these results, achieving MAE = 5.31, MAPE = 16.01%, and RMSE = 12.81.

W&B Project Link: [link](#)

Runs include all baseline and engineered models, alongside three sweep runs (deep-sweep-1, magic-sweep-2, sandy-sweep-3) used for hyperparameter optimization of the final Gradient Boosting model



Name	Status	Notes	User	Tags	Created	Runtime	Steps	Alpha	Features	Fit	Metrics	Method	Notes
baseline	Completed	Added to baseline	rahul		1m ago	1m 45s	10000	0.1	baseline	True	MAE: 5.31, RMSE: 12.81, MAPE: 16.01%	Gradient Boosting	Champion Model
engineered	Completed	Added to baseline	rahul		1m ago	1m 45s	10000	0.1	engineered	True	MAE: 5.31, RMSE: 12.81, MAPE: 16.01%	Gradient Boosting	
deep-sweep-1	Completed	Added to baseline	rahul		11m ago	1m 45s	10000	0.1	engineered	True	MAE: 5.31, RMSE: 12.81, MAPE: 16.01%	Gradient Boosting	
magic-sweep-2	Completed	Added to baseline	rahul		12m ago	1m 45s	10000	0.1	engineered	True	MAE: 5.31, RMSE: 12.81, MAPE: 16.01%	Gradient Boosting	
sandy-sweep-3	Completed	Added to baseline	rahul		13m ago	1m 45s	10000	0.1	engineered	True	MAE: 5.31, RMSE: 12.81, MAPE: 16.01%	Gradient Boosting	